

Smart Wearable Navigation Assistant

Learning Roadmap for Vision, Robotics, Sensor Fusion & Embedded Systems

Project Architecture

Camera + Depth → Environment Understanding → Obstacle / Stairs / Ground / Drop-off Detection
→ IMU Movement Estimation → Sensor Fusion → Risk Estimation → Haptic + Audio Feedback

1. Core Learning Roadmap

Stage	What to learn	Target depth
1. Computer Vision	OpenCV, NumPy, image geometry, calibration, perspective, optical flow	Strong
2. Deep Learning Vision	CNNs, object detection, segmentation, YOLO, lightweight models	Strong
3. Depth & 3D Vision	Depth maps, stereo/depth cameras, point clouds, 3D coordinates, geometry	Very strong
4. Sensor Fusion	IMU, coordinate frames, complementary/Kalman filters, EKF, synchronization	Very strong
5. Robotics	Transforms, localization, free space, occupancy grids, path planning, avoidance	Strong
6. Embedded Linux	Raspberry Pi, Linux, GPIO, I ² C, SPI, UART, processes, threads	Strong
7. Real-Time Systems	Latency, FPS, timestamps, asynchronous pipelines, profiling	Very strong
8. Haptics	Vibration motors, PWM, drivers, vibration encoding	Medium–strong
9. Audio	Spatial audio concepts, Bluetooth/AirPods integration, warning prioritization	Medium
10. Systems Engineering	Power, thermal limits, failure handling, reliability, testing	Very strong

2. Priority Tier 1 — Computer Vision

OpenCV + NumPy: image representation, coordinate systems, thresholding, morphology, contours, perspective transformation, camera calibration, lens distortion, optical flow, resizing/cropping, and geometric reasoning.

Engineering standard: do not merely memorize OpenCV functions. Be able to explain why each transformation improves the downstream perception problem and how it can fail.

3. Deep-Learning Computer Vision

Learn CNNs, feature maps, convolution, receptive fields, classification, object detection, semantic segmentation, instance segmentation, IoU, precision/recall, NMS, confidence thresholds, transfer learning, quantization, and lightweight deployment.

Your target is to understand failure modes such as dark scenes, motion blur, small objects, partial occlusion, and false detections—not simply adjust a confidence threshold.

4. Depth & 3D Vision — Major Upgrade

Learn the camera model and the mapping from pixel coordinates and depth to 3D coordinates:

$\text{pixel } (u, v) + \text{depth } Z \rightarrow \text{camera-frame point } (X, Y, Z)$

Topics: intrinsic/extrinsic parameters, stereo vision, depth cameras, point clouds, plane detection, ground-plane estimation, obstacle distance, surface normals, and 3D bounding geometry.

The goal is to move from 'a chair is detected' to 'the chair is 1.2 m away, 0.7 m right of the user, and the collision risk is increasing.'

5. IMU & Sensor Fusion

Because you already have MPU6050 experience, go beyond basic IMU usage. Study coordinate frames, rotation matrices, Euler angles, quaternions, gyro integration, accelerometer limitations, bias, drift, calibration, complementary filters, Kalman filters, EKF, and timestamp synchronization.

Research direction: camera + IMU fusion. The difficult part is combining measurements with different sampling rates, coordinate systems, noise characteristics, and latency.

6. Robotics

Learn coordinate transformations, homogeneous transformation matrices, rigid-body transformations, localization, occupancy grids, free-space representation, obstacle avoidance, A*, Dijkstra, potential fields, and the basic purpose of SLAM.

Do not start by blindly copying a ROS project. First understand the data flowing through the system; then use ROS 2 as an architecture and communication tool.

7. Embedded Linux & Real-Time Architecture

Learn Linux terminal usage, processes, threads, sockets, serial communication, I²C, SPI, UART, GPIO, camera interfaces, and Python/C++ interaction.

Your eventual system may run different rates simultaneously:

Camera 30 FPS | YOLO 10-20 FPS | Depth 30 FPS | IMU 100+ Hz | Haptics real-time | Audio asynchronous

Study producer/consumer queues, timestamps, synchronization, dropped frames, CPU/GPU bottlenecks, latency measurement, and profiling.

8. Haptic + Audio Interaction

Learn vibration motor drivers, PWM, safe electrical driving, and how to encode direction, distance, and urgency into vibration patterns. For audio, study spatial-audio concepts, Bluetooth integration, and warning prioritization.

The research question is not 'How many warnings can I generate?' It is 'How much information can the user interpret without cognitive overload?'

9. Research-Level Layer: Risk Estimation

Move beyond object detection. Combine distance, relative velocity, object type, trajectory, and user movement into a risk estimate.

Example conceptual model: $\text{risk} = f(\text{distance, relative velocity, object type, trajectory, user motion})$

This is where perception becomes decision-making rather than a collection of independent AI demos.

10. Recommended Learning Order

OpenCV → Camera Calibration → Deep Learning → YOLO → Segmentation → Depth / 3D Vision → Coordinate Transformations → IMU → Sensor Fusion → ROS 2 / Robotics Architecture → Real-Time Systems → Haptic + Audio Interface → Risk Estimation → Full Wearable Prototype

11. First Serious Challenge — Before Buying Hardware

Implement the perception pipeline on your Mac using ordinary RGB video and simulated or recorded depth.

For every detected obstacle, estimate: object type, distance, left/right position, whether it is approaching or stationary, and a collision-risk level.

This exercise will expose whether you actually understand computer vision, 3D geometry, and perception architecture instead of only being able to follow tutorials.

Professional mindset: Treat the project as a safety-critical perception system. Measure latency, false positives, false negatives, failure cases, environmental limitations, and user cognitive load. A demo that works once is not the same as a reliable engineering system.